DNN Programming Assignment: RNN, LSTM, GRU and Transformer

Student Name: Umesha Margi
Student ID: 2026AJ05949

Problem 1: RNN, LSTM, and GRU on Sequences

1.1 Data Pipelines & Setup

- **UCI HAR Dataset:** Standardized per-channel using statistics (μ, σ) computed strictly from the training split. Validation sets were formed by holding out 4 out of 21 training subjects per seed (np.random.default_rng). Class balance across all 6 classes was verified (**Figure 1: class_balance.png**).
- **Adding Problem Pipeline:** Verified synthetic sequence generator for lengths $T \in \{50, 100, 200, 400\}$. Unit tests confirmed that every generated sequence contains exactly two marker indicators ($x_{t,1} = 1$)—one in $[0, T/2)$ and one in $[T/2, T)$—and that target y equals the exact sum of the marked values.

1.2 Cell Verification & Parameter Derivations

Hand Derivations ($d = 9, h = 64$)

Vanilla RNN Cell:

$$P_{RNN} = h \cdot d + h \cdot h + h = 64 \times 9 + 64 \times 64 + 64 = 576 + 4096 + 64 = 4,736$$

- **LSTM Cell:** Contains 4 gates (input, forget, cell candidate, output):

$$P_{LSTM} = 4 \times (h \cdot d + h \cdot h + h) = 4 \times (576 + 4096 + 64) = 18,944$$

- **GRU Cell:** Contains 3 gates (reset, update, new state candidate):

$$P_{GRU} = 3 \times (h \cdot d + h \cdot h + h) = 3 \times (576 + 4096 + 64) = 14,208$$

PyTorch Reconciliation

PyTorch reports parameter counts of **4,800** (RNN), **19,200** (LSTM), and **14,400** (GRU).

1. **Reason:** PyTorch maintains two separate bias vectors—an input-to-hidden bias (b_{ih}) and a hidden-to-hidden bias (b_{hh})—per gate for hardware/CUDA matrix operation

optimization.

2. **Difference:** $+ 1 \times h$ per gate ($+ 64$ for RNN, $+ 256$ for LSTM, $+ 192$ for GRU).
3. **Unit Test Result:** All custom cells (MyRNNCell, MyLSTMCell, MyGRUCell) matched PyTorch reference implementations within 1×10^{-5} tolerance on random test batches.

1.3 UCI HAR Classification Results

Evaluated across 3 fixed random seeds using Adam ($lr = 10^{-3}$) for 15 epochs per model.

Model	Test Accuracy (%)	Macro-F1 Score	Parameter Count	Training Time (sec/epoch)
Vanilla RNN	82.4 ± 1.8	0.812 ± 0.021	4,800	0.38
LSTM	91.8 ± 0.6	0.914 ± 0.007	19,200	0.42
GRU	92.3 ± 0.4	0.921 ± 0.005	14,400	0.40

*(The confusion matrix of the best performing model, GRU, is shown in **Figure 2: har_confusion_matrix.png**).*

1.4 Adding Problem Results

Target constant-prediction baseline MSE ≈ 0.167. A run is declared solved if test MSE < 0.05.

Sequence Length (T)	Model	Test MSE (Mean±Std)	Solved Runs (/3)
50	Vanilla RNN / LSTM / GRU	0.174 ± 0.003 / 0.004 ± 0.001 / 0.003 ± 0.001	0 / 3 / 3
100	Vanilla RNN / LSTM / GRU	0.172 ± 0.002 / 0.012 ± 0.004 / 0.008 ± 0.003	0 / 3 / 3
200	Vanilla RNN / LSTM / GRU	0.169 ± 0.001 / 0.045 ± 0.018 / 0.021 ± 0.009	0 / 2 / 3
400	Vanilla RNN / LSTM / GRU	0.168 ± 0.001 / 0.098 ± 0.035 /	0 / 0 / 1

		0.058 ± 0.021	

*(Learning curves at $T = 200$ are plotted in **Figure 3: adding_curves_T200.png**; Test MSE vs. T is plotted in **Figure 4: adding_mse_vs_T.png**).*

1.5 Gradient Analysis & Experiments

(a) Gradient Norm Analysis ($\| \partial L/\partial h_t \|$ vs t)

As sequence length grows to $T = 100$, backpropagating gradients through Vanilla RNN hidden states exponentially decay ($\| \partial L/\partial h_t \| \propto \lambda^{T-t}$ where $\lambda < 1$) (**Figure 5: gradient_norm.png**). In contrast, LSTM and GRU retain constant non-zero gradient magnitudes across early time steps due to their additive cell state updates and gating mechanisms.

(b) Hidden-Size Sweep $\{16, 32, 64, 128\}$ on HAR

Increasing hidden size improves accuracy up to $h = 64$ ($87.1\% \rightarrow 92.3\%$ for GRU), while $h = 128$ yields marginal gains (92.7%) at double the parameter cost (**Figure 7: hidden_sweep.png**).

(c) LSTM Forget-Gate Bias Ablation ($T = 200$)

Initializing the forget-gate bias $b_f = 1.0$ vs 0.0 at $T = 200$ reduces test MSE from 0.162 to 0.012 (**Figure 6: ablation_forget_bias.png**). Setting $b_f = 1.0$ forces initial forget gate activation $f_t \approx \sigma(1.0) \approx 0.73$, preventing the network from erasing early sequence information during initial training iterations.

1.6 One-Page Analysis & Synthesis

- **Gradient Behavior & Long-Term Memory:**
  The vanishing gradient problem heavily impacts the Vanilla RNN. In the adding problem, the RNN fails across all sequence lengths ($T \geq 50$), remaining stuck near the constant-prediction baseline of 0.167. Tracking $\| \partial L/\partial h_t \|$ reveals that recurrent multiplication by weight matrix W_{hh} shrinks gradients to zero before reaching the first marked token at $t < T/2$. gated architectures (LSTM and GRU) establish constant-gradient paths that preserve information across long time horizons.
- **Model Failure Horizons as T Grows:**
  As sequence length reaches $T = 400$, even gated models begin to experience stability issues. LSTM solved 0/3 runs and GRU solved 1/3 runs without gradient clipping, as long sequence rollouts accumulate numerical instabilities and exploding gradient spikes.
- **Trade-Off Analysis (Accuracy vs. Parameters vs. Speed):**
  - **Vanilla RNN:** Low parameters (4,800) and fastest speed (0.38 s/epoch), but poor accuracy (82.4%) and inability to solve long-memory tasks.
  - **LSTM:** High accuracy (91.8%) and strong memory retention, but requires 19,200 parameters and 0.42 s/epoch.

- **GRU:** Achieves the highest accuracy (92.3%) and fast convergence, using 14,400 parameters (25% fewer than LSTM) and running at 0.40 s/epoch.

- **Recommendation for Battery-Powered Wearables:**
  GRU is the recommended architecture for battery-powered wearable devices. It matches or exceeds LSTM accuracy on sensor classification tasks (92.3% vs. 91.8%) while requiring **25% fewer parameters and matrix operations** per step. This reduction in parameter count significantly lowers memory bandwidth demands and energy consumption during continuous on-device inference.

Problem 2: Transformer Built From Scratch

2.1 Attention & Parameter Derivations

Attention Unit Tests

Scaled Dot-Product Attention: Implemented directly from matrix operations:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Assert tests verified that every row of the attention matrix sums to 1.0 and matches torch.nn.functional.scaled_dot_product_attention within 1×10^{-5} tolerance.
- **Causal Masking:** Verified that applying lower-triangular causal masks sets all upper-triangular attention weights for future position steps strictly to 0.0.

Hand Derivation of Encoder Block Weights ($d = 128$, $d_{ff} = 512$)

- **Multi-Head Attention (MHA):** $W_Q, W_K, W_V, W_O \in \mathbb{R}^{d \times d} \implies 4 \times (d \cdot d) = 4d^2$.
- **Feed-Forward Network (FFN):** Linear 1 ($d \cdot d_{ff}$) + Linear 2 ($d_{ff} \cdot d$) = $2 \cdot d \cdot d_{ff}$.

  Pure Weight Parameter Count:

$$4d^2 + 2d \cdot d_{ff} = 4(128^2) + 2(128)(512) = 65,536 + 131,072 = 196,608$$

Reconciliation with Code (Biases & LayerNorms)

4. Attention Projection Biases (W_Q, W_K, W_V, W_O): $4 \times d = 512$ parameters.
5. FFN Biases (Linear 1 + Linear 2): $d_{ff} + d = 512 + 128 = 640$ parameters.
6. Two Pre-LayerNorm Modules (γ, β $\in R^d$ each): $2 \times (2d) = 512$ parameters.
7. **Total Parameters per Block:** $196,608 + 512 + 640 + 512 = 198,272$ parameters.

2.2 Task A: HAR Encoder Classifier & Positional Encodings

Evaluated on the UCI HAR sequence dataset using a Transformer Encoder architecture (1 block, $d = 128$, 4 heads, $d_{ff} = 512$) with a CLS classification token.

Positional Encoding Variant	Test Accuracy (%)	Macro-F1 Score	Parameter Count
None (No Position Info)	84.1 ± 1.2	0.835 ± 0.015	412,422
Learned Positional Embeddings	90.5 ± 0.8	0.901 ± 0.009	428,806
Sinusoidal Positional Encoding	91.2 ± 0.5	0.910 ± 0.006	412,422

(Positional encoding ablation results are saved in **Figure 8: positional_encoding_ablation.png**).

2.3 Task B: Character-Level Causal Language Model

Trained on Tiny Shakespeare (1.1 MB, 65 distinct characters) with context length 128, $d = 128$, 4 heads, and 4 encoder blocks.

- **Validation Loss: 1.64 nats** (Uniform baseline guess: $\ln(65) \approx 4.17$ nats).
- **Bits Per Character (BPC):** $\frac{1.64}{\ln(2)} \approx 2.366 BPC$.
- **Learning Trajectory:** Training/validation cross-entropy loss trajectories are saved in **Figure 9: transformer_learning_curve.png**.

Sampling Analysis Across Temperatures (Top-$k = 10$)

- **Temperature 0.2 (Low Entropy):** Produces highly repetitive, grammatically consistent text with standard words and common punctuation (e.g., *"KING RICHARD: I will not go to the court..."*).
- **Temperature 0.8 (Optimal Entropy):** Balances vocabulary diversity and syntactic coherence, generating realistic Shakespearean dialogue structures.
- **Temperature 1.5 (High Entropy):** Generates noisy, ungrammatical character sequences and nonexistent words due to flattened output probabilities.

2.4 Analysis and Ablations

- **Attention Map Specialization:** Heatmaps for Head 1 (**Figure 10:**

attention_heatmap_head1.png) and Head 2 (**Figure 11: attention_heatmap_head2.png**) demonstrate specialization: Head 1 attends strictly to immediately adjacent tokens (local syntactic dependence), whereas Head 2 captures long-range dependencies across clauses.

- **Multi-Head Sweep (Fixed $d = 128$):** Increasing heads from 1 to 4 improves validation loss from 1.82 to 1.64 nats, while moving to 8 heads yields diminishing returns (1.61 nats) (**Figure 12: heads_ablation.png**).
- **Causal Mask Ablation:** Removing the causal mask during LM pretraining causes validation loss to degrade to **3.85 nats** (**Figure 13: no_mask_result.png**), as the model learns to cheat during training by attending to future tokens, completely breaking autoregressive text generation.
- **Context Length Scaling ($O(n^2 \cdot d)$):** Measuring runtime per step (**Figure 14: context_time.png**) and peak memory footprint (**Figure 15: context_memory.png**) across context lengths $n \in \{64, 128, 256, 512\}$ confirms quadratic scaling due to full $n \times n$ attention matrix computations.